System Requirements Specification (SRS)

Project Name: *DIGITAL FARMERS' PRODUCE SYSTEM: A CLOUD-BASED PLATFORM FOR OPTIMIZING CROP MANAGEMENT AND MARKET LINKAGES*

Version 1.0

Prepared by: Bryan Malinao
Date: 2025-12-17

Revision History

Maintain a record of document revisions here.

Date	Version	Description / Changes Made
2025-09-23	1.0	Initial draft
2025-12-31	1.1	Language Selection and SMS Notification

Contents

List of Figures

1 Introduction

1.1 Purpose

To develop a cloud-based platform that optimizes seasonal crop management and strengthens market linkages among farmers in Barangay Sinacaban, thereby improving productivity, reducing post-harvest losses, and increasing income through digital connectivity. This SRS is intended for:

- Development team – to design and implement the system
- QA testers – to validate features against acceptance criteria
- Business stakeholders (Barangay LGU, Department of
- Agriculture officers) – to ensure alignment with program goals
- User representatives (farmers and buyers) – to verify fitness for use

1.2 Scope

In Scope

- Farmer product posting & availability scheduling.
- Buyer discovery & direct messaging to farmers.
- Seasonal calendar; price/volume trend snapshots.
- Stock monitoring.
- Role-based access (Farmer, Buyer, Agri Officer, Admin).
- Decision support (basic analytics and trend reports).
- Notifications (in-app, optional SMS/e-mail).
- Audit logs and basic governance dashboards.

Out of Scope

- Online payments and escrow.
- Logistics/delivery management.
- Complex price forecasting or advanced ML models (beyond basic trend analytics).
- Non-agricultural barangay services (HR, payroll, etc.).

1.3 Definitions, Acronyms, and Abbreviations

- NFR: Non-Functional Requirement
- DFPS – Digital Farmers' Produce System
- DA – Department of Agriculture
- LGU – Local Government Unit
- PWA – Progressive Web App
- UI/UX – User Interface / User Experience
- API – Application Programming Interface
- NFR – Non-Functional Requirement
- RBAC – Role-Based Access Control

1.4 References

2 Overall Description

2.1 Product Perspective

DFPS is a cloud-hosted, multi-tenant web application with a PWA front end. It integrates three stakeholder groups:

- Farmers: post produce, harvest windows, quantities, and prices; request resources.
- Buyers: search by crop, date, quantity, and location; contact farmers.
- Agri Officers: monitor barangay-level stocks, respond to inquiries, manage aid/subsidies, and publish advisories.

2.2 Product Functions

- Authentication & RBAC (Farmer, Buyer, Agri Officer, Admin)
- Farmer inventory & seasonal scheduling
- Buyer search & contact
- Announcements/advisories (Agri Officer)
- Subsidy/resource request intake and tracking
- Decision support dashboards (stock trends, demand/price snapshots)
- Notifications and reminders
- Audit logging & basic governance metrics

2.3 User Classes and Characteristics

- **Farmer**: Low to moderate digital literacy; needs simple, mobile-first forms; intermittent connectivity. The system provides Cebuano language support and SMS notifications for important announcements and DA subsidies, ensuring farmers stay informed even on basic phones.
- **Buyer** (retailers, traders, institutions): Needs fast search and filters, reliability indicators, and contact options; compatible with smartphones and low-end devices.
- **Agri Officer**: Needs monitoring, reporting, and approval workflows; familiar with desktop interfaces; can send SMS alerts to farmers.
- **Admin**: Responsible for system configuration, user management, and data quality oversight.

2.4 Operating Environment

- **Hosting/Deployment**: GoDaddy shared/cloud hosting with HTTPS/SSL and managed domain.
- **Runtime**: PHP 8.x, Flight microframework, Composer.
- **Database**: MySQL (GoDaddy-managed instance).
- **Client**: Bootstrap 5 responsive UI; latest Chrome, Edge, Safari, and Firefox; Android/iOS browsers.
- **Interfaces**: REST-like endpoints over HTTPS; server-rendered HTML and form posts.
- **UI/UX & PWA Features**:
  - Progressive Web App (PWA) – accessible via web or mobile browsers without installation.
  - Mobile-first, touch-friendly design built with Bootstrap 5 for consistency across devices.

2.5 Constraints

Data Privacy Act (PH) compliance; minimal PII. Shared hosting constraints (no long-running daemons). Intermittent connectivity expected; server-generated pages preferred.

3 Functional Requirements

FR-01: Authentication and RBAC
The system shall provide a login with username or password, enforce CSRF protection, and restrict access by role (farmer, buyer, da officer).
Priority: Must have
Acceptance Criteria: When a transaction is flagged, an alert appears on the dashboard in 5 seconds.

FR-02: Product Categories
The system shall maintain a list of standard product categories used for product filtering and classification.
Priority: Must have
Acceptance Criteria: The category list loads on forms and filters; category names are unique.

FR-03: Farmer Listings (CRUD)
The system shall allow farmers to create, edit, delete, and view product listings including name, description, category, quantity, unit, price, image, status, and posted date.
Priority: Must have
Acceptance Criteria: CSRF-protected CRUD succeeds; farmers only see their listings; images are saved and referenced; status updates appear immediately.

FR-04: Buyer Search and Discovery
The system shall provide searching and filtering of product listings with text query, category, price range, sorting, and pagination (12/24/48).
Priority: Must have
Acceptance Criteria: Results reflect filters and sorting; P95 response time is 2 seconds or less on 50k listings.

FR-05: Messaging
The system shall allow users to send and receive messages with subject and body in an inbox interface.
Priority: Must have
Acceptance Criteria: Messages appear in the receiver's inbox within 5 seconds; only participants can view conversations; unread/read flags update.

FR-06: Resource Requests
The system shall allow farmers to submit resource requests and DA officers to approve or reject them with remarks.
Priority: Should have
Acceptance Criteria: Status changes record decision date and processed by; request lists display current status.

FR-07: DA Dashboard and Metrics
The system shall provide DA officers with dashboard metrics for products, categories, and resource requests.
Priority: Should have
Acceptance Criteria: Dashboard loads within 3 seconds; metrics match database values.

FR-08: CSV Export (Products)
The system shall export product listings as a CSV file reflecting active filters.
Priority: Should have
Acceptance Criteria: CSV opens correctly in spreadsheet software; the row count matches the filtered results.

FR-09: Validation and CSRF

The system shall apply server-side validation and require CSRF tokens for all mutating actions.
Priority: Must have
Acceptance Criteria: Invalid inputs return meaningful errors; CSRF mismatches block actions.

FR-10: Flash Messaging and UX
The system shall provide clear success and error flash messages and use Bootstrap for forms and tables.
Priority: Should have
Acceptance Criteria: Users receive appropriate feedback after create, update, delete, and decision actions.

FR-11: SMS Notifications
The system shall send SMS alerts specifically for system announcements and Department of Agriculture (DA) subsidy notifications. This ensures that farmers without smartphones receive important updates even in areas with limited internet connectivity.
Priority: Should have
Acceptance Criteria: Users receive timely SMS messages for all official system announcements and DA subsidy updates.

4 Non-Functional Requirements

4.1 Performance

- P95 page render $\leq 3.0\,\text{s}$ on GoDaddy shared hosting; search query $P_{95} \leq 300\,\text{ms}$ DB time with proper indexes.
- CSV export streams without exhausting memory for up to 100k rows.

4.2 Security

- HTTPS only (TLS 1.2+); session cookies `HttpOnly`, `Secure`; CSRF on forms.
- Passwords hashed with PHP `password_hash` (Argon2id or bcrypt with strong cost).
- Input validation & output escaping (prevent SQLi/XSS); PDO prepared statements everywhere.
- Role checks server-side on every route.

4.3 Availability & Backup

- Target uptime 99.9% monthly.
- Daily DB backups via GoDaddy/MySQL; RPO $\leq 24\,\text{h}$, RTO $\leq 4\,\text{h}$.

4.4 Usability & Accessibility

DFPS is designed to be accessible and user-friendly across devices and user literacy levels:

- Screens use Bootstrap 5 components, clear form labels, keyboard navigation, and color contrast targeting WCAG 2.2 AA standards.
- Language toggle allows users to select Cebuano, ensuring local language accessibility.
- Icons, visual cues, and simplified labels reduce reliance on text.
- SMS notifications deliver system announcements and DA subsidy updates to users without smartphones.
- Mobile-first design optimized for low-end devices; supports touch input and intermittent connectivity.

4.5 Maintainability

- PSR-4 structure; Composer autoload; `.env` for DB creds; small, testable controllers/models.

5 External Interface Requirements

5.1 User Interface (Bootstrap 5 Views)

- **Auth:** `/login` (GET/POST), `/logout`
- **Dashboards:** `/dashboard` → role-based: farmer/buyer/da
- **Products (Farmer):** `/products`, `/products/create` (GET), `/products` (POST), `/products/{id}/edit` (GET), `/products/{id}/update` (POST), `/products/{id}/delete` (POST)
- **Requests:** `/requests` (role-aware), create (POST), decision (DA POST)
- **Messages:** `/messages` (inbox), send (POST)
- **Export:** `/export/products.csv`

5.2 API/Routes (Flight)

- Defined in `app/routes.php` using `Flight::route('METHOD /path', ...)`.
- JSON not required for current flows (server-rendered forms), but endpoints are REST-like.

5.3 Communications

- All endpoints behind HTTPS; optional email/SMS integration can be added later.

6 System Models

6.1 Infrastructure Architecture

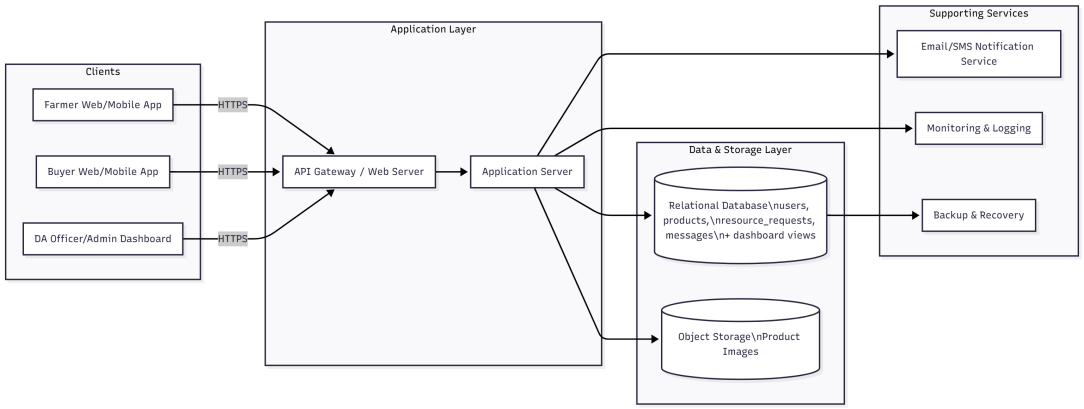

Figure 1: Infrastructure Architecture

Figure 1 shows how the Digital Farmers' Produce System is deployed and how users reach it. Farmers, buyers, and DA officers use web or mobile browsers to access the application over HTTPS. Their requests go through the web/application server, which handles sessions, validation, and business logic. This server reads and writes data to a MySQL database (for users, products, messages, and resource requests) and to a separate image storage layer for product photos and media. A backup and monitoring component regularly copies database and file data to backup storage and tracks system health. Overall, the diagram highlights that the system is cloud-hosted, centrally managed, and accessible to all stakeholders using standard browsers.

6.2 Entity Relationship Diagram (Logical View)

Figure 2 presents the logical data model of the system. The central *users* table stores all account information and is linked to several core entities. Each farmer can own many products, and each product belongs to a *product_categories* record for standard classification. *messages* connect one user as the sender and another as the receiver, capturing private communication between farmers, buyers, and DA officers. The *resource_requests* entity records farmers' requests for assistance, with DA officers linked as the processors of those requests. Summary or reporting views such as *v_available_by_category* and *v_resource_requests_metrics* provide aggregated information for dashboards, like total available quantity per category or counts of approved and pending requests. This ERD shows how the database supports inventory, communication, and monitoring in a structured way.

6.3 Use Case Diagram

Figure 3 summarizes what each role can do in the system. The farmer actor interacts with use cases related to product management (add, edit, and deactivate products), upload of product images and media, sending messages, receiving notifications, and submitting resource requests. The buyer actor can browse and search products, send messages to

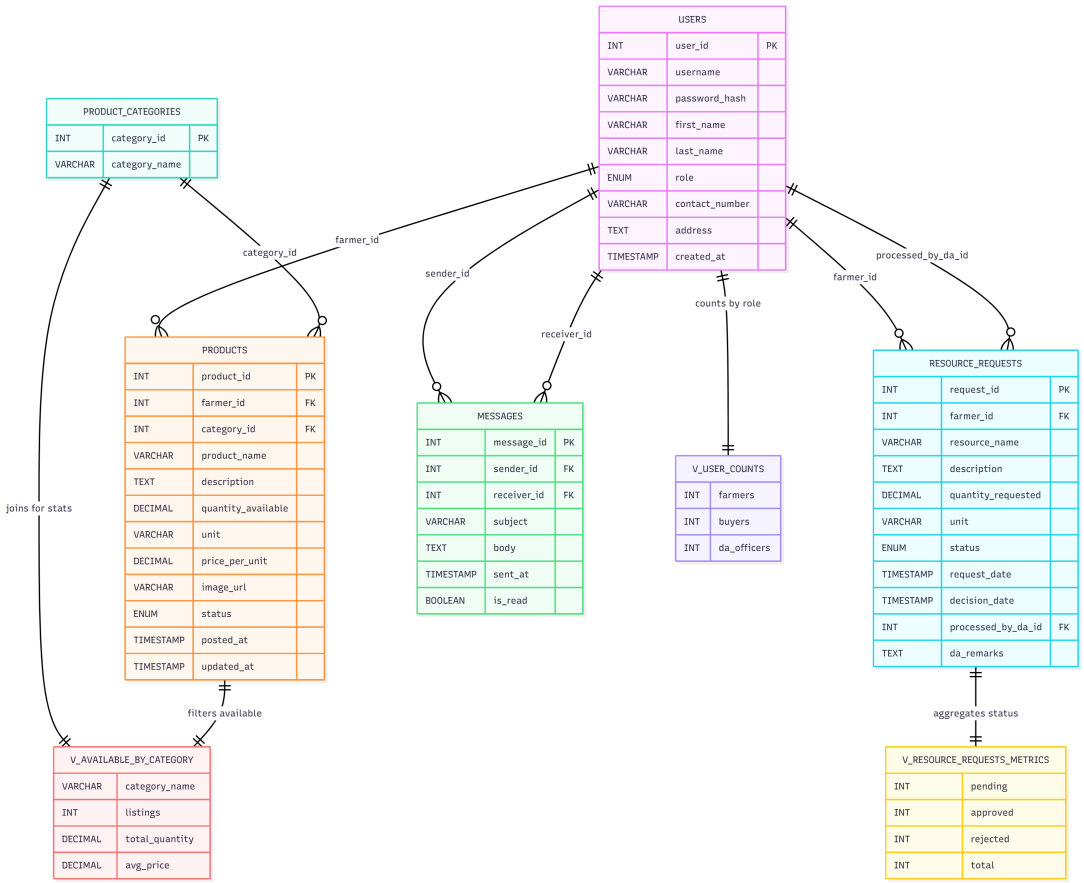

Figure 2: Entity Relationship Diagram (Logical View)

Figure 3: Use Case Diagram

farmers, and receive responses and notifications. The Department of Agriculture Sinacaban actor focuses on oversight and support: they manage resource requests, respond to messages, send announcements or notifications, and generate reports or CSV exports. Together, these use cases show that the system covers the full cycle from posting produce and discovering products to coordination with the DA and generation of monitoring reports.

6.4 Entity Relationship Diagram (Physical / Detailed View)

Figure 4 shows a more detailed, implementation-oriented ERD. It includes explicit primary keys, foreign keys, data types, and status or timestamp fields for each table. For example, *products* includes fields such as *product_name*, *quantity_available*, *unit_price*, *status*, and *posted_at*, all linked to a *farmer_id* referencing *users*. *messages* stores subject, body, read status, and created timestamps for each conversation between users. *resource_requests* tracks the type of resource, quantity requested, status, decision date, and the DA officer who processed the request. By specifying these attributes and constraints, the diagram guides actual database creation and ensures that future features (like reporting and validation) have the necessary data.

6.5 Context Diagram (Level 0)

Figure 5 gives a high-level context view of the Digital Farmers' Produce System as a single process interacting with external entities. Farmers send data such as registration details, product posts, images, messages, and resource requests into the system. Buyers provide

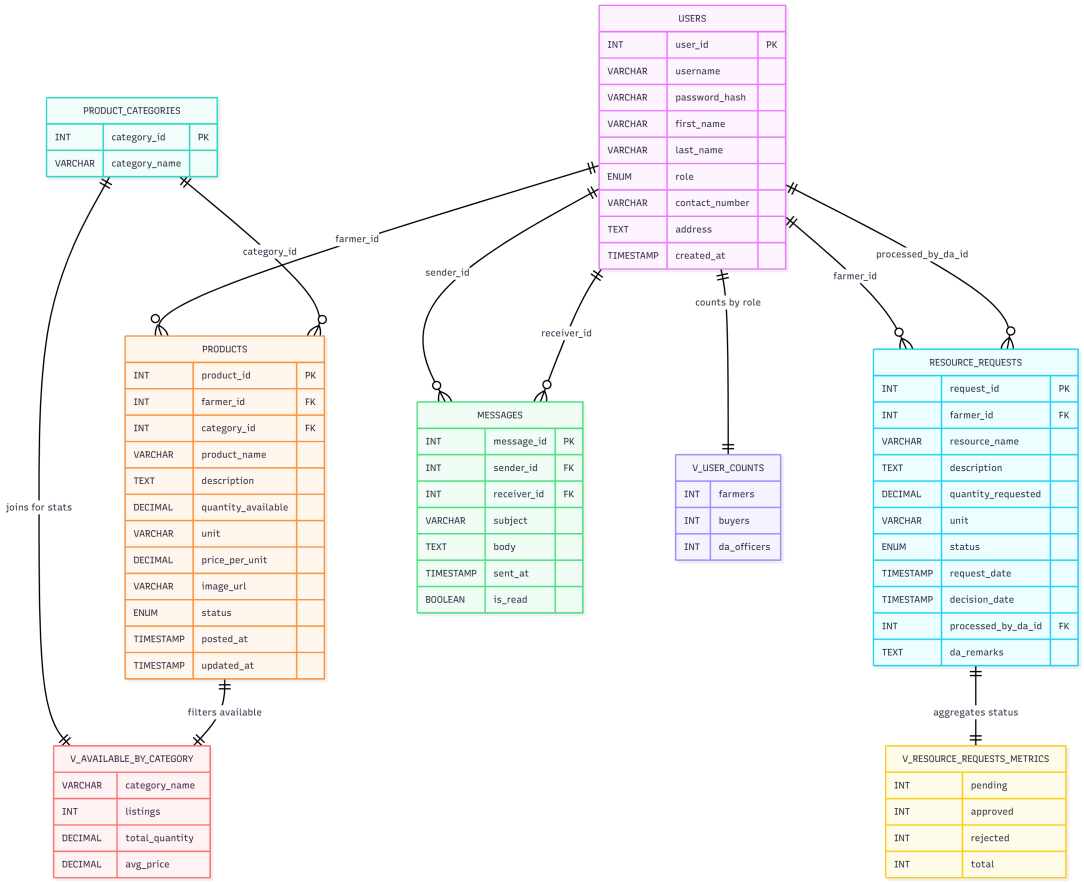

Figure 4: Entity Relationship Diagram

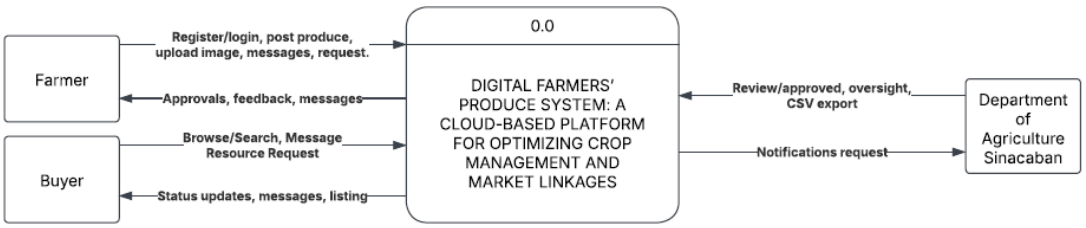

Figure 5: Context Diagram (Level 0)

registration information, browsing and search requests, and messages. The Department of Agriculture Sinacaban sends review decisions, oversight inputs, and notification content, and in return receives CSV exports, status summaries, and feedback from the system. This diagram clearly defines the system boundary: everything inside the central DFPS block is part of the software, while farmers, buyers, and the DA office are external actors exchanging information with it.

6.6 Data Flow Diagram (Level 1)

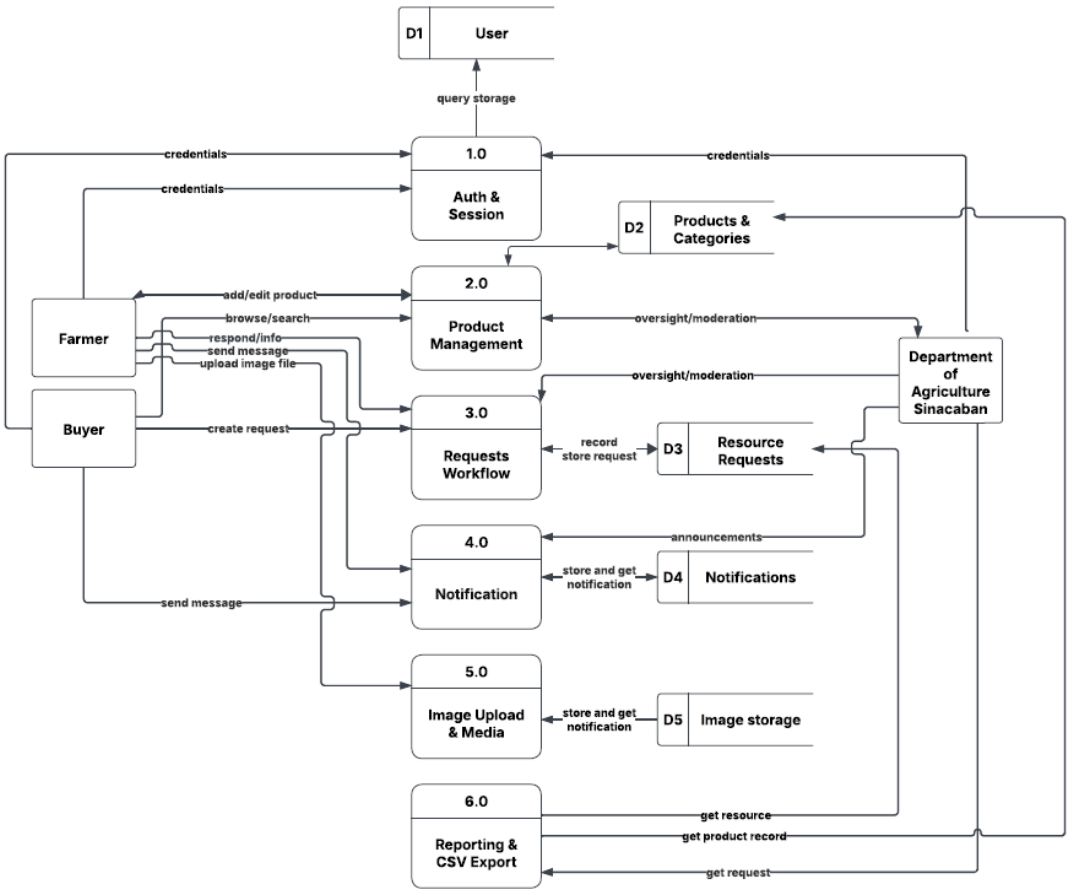

Figure 6: Data Flow Diagram (Level 1)

Figure 6 decomposes the system into major internal processes and data stores. The 1.0 Auth & Session process validates user credentials and manages sessions using the *User* data store. 2.0 Product Management handles adding, editing, and deactivating products, working with the *Products & Categories* data store. 3.0 Requests Workflow and *D3 Resource Requests* manage the life cycle of farmers' resource requests, from submission to approval or rejection by DA officers. 4.0 Notification and *D4 Notifications* are responsible for recording and delivering announcements and alerts to users. 5.0 Image Upload & Media interacts with *D5 Image Storage* to store and retrieve product images. Finally, 6.0 Reporting & CSV Export reads from the relevant data stores to generate reports and downloadable CSV files. The flows between farmers, buyers, DA officers, and these processes show how information moves through the system from login up to reporting.

7 Assumptions and Dependencies

Capture expectations and external dependencies.

Example: Assumes POS system provides real-time data. Depends on AWS hosting and Twilio for SMS alerts.

8 Appendices

8.1 Glossary

Add detailed definitions of terms used in the document.

8.2 Traceability Matrix

Map each requirement to its design element and test case.

Example:

Requirement ID	**Design Reference**	**Test Case ID**
FR-01	Login Module Design Doc	TC-01: Verify login with valid credentials
FR-02	Fraud Detection Algorithm Spec	TC-05: Trigger alert for flagged transaction

8.3 Change History

Track changes to the SRS document over time.